

repita, agora com esse anexo que contem as imagens

DeepSeek AI

思考：

We need to process the user's request: they provided a PDF file "Emacs afazeres.pdf" which contains text and images (but as text layer, we have the OCR content). The user wants to repeat the previous task but with this new attachment that includes images. Essentially, they want an analysis of the errors shown in the images (now embedded in PDF as text), and generate a structured prompt for corrections and upgrades. The previous response was a detailed prompt. Now they want me to "repita, agora com esse anexo que contem as imagens" - so repeat the same style but using the new content from the PDF.

I need to extract the relevant error messages and requests from the PDF text. The PDF pages contain a mix of terminal output, error messages, and user comments. I should interpret the images (though the text layer provides some of it). The user wants an optimized prompt for OpenCode to fix issues.

Let me parse the PDF content page by page:

Page 1:

- Vterm needs vterm-module to work. Compile it now? (y or n) - user says even after choosing y, problem repeats.
- Error installing bookmark+ and loading init.el.
- Requests: toolbar with large icons, toggle visibility via macros; customize start page like desktop with multiple desktops, contexts, projects, dashboard, updated info, notifications.
- Reopened with --debug-init, showed screens (images referenced but not shown in text).

Page 2: just a number "1" (probably image placeholder)

Page 3:

- dpkg error: requested operation requires superuser privilege (running apt without sudo? but in proot-distro, should be root? Actually proot-distro runs as user, but apt needs sudo? Might be permission issue)

- Configuring and compiling pdf-tools: ./configure line 4925 syntax error near unexpected token 'png', PKG_CHECK_MODULES(png, libpng) - missing pkg-config or libpng-dev.
- Build failed.
- Debugger entered: Lisp error: end-of-file "/home/sukata/.emacs.d/init.el" - init.el not properly closed? read error.
- Error: wrong-type-argument listp "-/.emacs.d/autosaves/" - path starts with dash? Possibly typo.

Page 4:

- Installing packages: dpkg error again requiring superuser privilege.
- Build fails similarly.
- User accessed dired (M-x dired) and icons appeared, but when accessing via SSH from desktop, icons did not appear.

Page 5: garbled text (likely icon glyphs) - not useful.

Page 6: blank? Actually page 6 has no text.

Page 7:

- Both dired and treemacs: when trying to open any file, error: Wrong type argument: listp, "-/.emacs.d/autosaves/"
- User asks how to remove Neotree frame/window/buffer when not needed. Example: opened Neotree, split vertically, tried to open file got error, then dired opened in right side instead of full screen, even C-x 1 didn't work.
- Tried macro C-c C-p p (MD->PDF) didn't work. Justify and consider other macros may not work. Request to test all.
- Verify warnings about missing character U+1F7E1 (yellow circle) in font DejaVu Sans.
- Gap analysis of files in /home/sukata/emacs (list of PDFs and MDs).
- Chatlog confusing, wants suggestions for organizing chatlogs/sections/configurations in OpenCode.
- Implemented FUTURE_IMPROVEMENTS.md.

- Suggestion to move parts of init.el to elisp modules.
- Copied another init.el from desktop (initAntiGrav.el) to integrate.
- Read and gap analysis of session-ses_295b.md and emacsproj.md.
- Syncting between desktop and termux proot-distro: installed Android app but can't access proot-distro folder; suggests symlink or CLI option.
- Request to perform tests (unit, integration, system, regression, end-to-end) with fixtures and test doubles.
- After corrections, update emacs-manual.md.

Page 8: continues list of files and repeats some requests.

Now I need to produce a similar structured prompt as before, but using this new information. The previous response was a comprehensive prompt. I'll adapt it, focusing on the specific errors shown: vterm compilation, bookmark+ error, init.el syntax/EOF, autosaves path error, pdf-tools build failure (missing pkg-config, libpng), permission issues (sudo in proot), SSH icon missing, Neotree window management, macro failures, font warnings.

Also address the organizational questions about chatlogs and OpenCode.

I'll produce a response that repeats the style: emoji outline, rationale, step-by-step plan, test requirements, etc.

Let me write.

Prompt de Engenharia de Correção e Upgrades – Emacs no Termux/Proot-Distro (com base no PDF anexo)

Com base no arquivo `Emacs afazeres.pdf` (contendo screenshots e logs de erro), estruturei um prompt otimizado para o OpenCode, focado em corrigir **todas as falhas identificadas** e implementar as **melhorias solicitadas** de forma incremental, testável e documentada.

Análise de Erros por Página do PDF

Página	Erro / Sintoma	Causa Provável
1	<code>`term`</code> precisa do módulo, compilação falha mesmo respondendo “y”	<code>`term`</code> requer <code>`libvterm`</code> e compilação com suporte a módulos dinâmicos; ambiente Termux/proot pode não ter ferramentas de build (<code>`make`</code> , <code>`gcc`</code> , <code>`cmake`</code>).
1	Erro ao instalar <code>`bookmark+`</code> e ao carregar <code>`init.el`</code>	Dependência faltante ou conflito de versão; <code>`init.el`</code> pode ter sintaxe inválida ou chamada a pacote não instalado.
3	<code>`dpkg: error: requested operation requires superuser privilege`</code>	Dentro do proot-distro, o usuário não tem <code>`sudo`</code> configurado; necessário usar <code>`sudo`</code> ou rodar como root.
3	<code>./configure: line 4925: syntax error near unexpected token 'png'`</code> (pdf-tools)	Pacotes de desenvolvimento ausentes: <code>`libpng-dev`</code> , <code>`pkg-config`</code> , <code>`libpoppler-dev`</code> .
3	<code>`Debugger entered: Lisp error: end-of-file "/home/sukata/.emacs.d/init.el"``</code>	Arquivo <code>`init.el`</code> com fechamento de parênteses incorreto ou incompleto.
3, 7	<code>`Wrong type argument: listp, "-/.emacs.d/autosaves/"`</code>	Caminho inválido (começa com <code>`-`</code>); provavelmente erro de digitação na variável <code>`auto-save-list-file-prefix`</code> ou similar.
4	Ícones no <code>`dired`</code> funcionam localmente, mas não via SSH	Terminal remoto não suporta Nerd Fonts ou <code>`all-the-icons`</code> não detecta a fonte correta.
7	Ao tentar abrir arquivo no <code>`dired`/`treemacs`</code> → mesmo erro <code>`listp`</code>	O erro de autosaves corrompe a abertura de arquivos.
7	<code>`Neotree`</code> divide tela verticalmente e não fecha; <code>`C-x 1`</code> não maximiza	Conflito entre <code>`neotree`</code> e <code>`treemacs`</code> ; lógica de gerenciamento de janelas inconsistente.
7	Macro <code>`C-c C-p`</code> (MD → PDF) não funciona	<code>`pandoc`</code> ou <code>`texlive`</code> não instalados; ou <code>`org-mode`</code> sem exportador LaTeX configurado.
7	Warnings: <code>`Missing character: There is no`</code> 🟡 (U+1F7E1) in font DejaVu Sans`	Fonte <code>`DejaVu Sans`</code> não cobre emojis; instalar <code>` noto-fonts-emoji`</code> e configurar fallback.
8	Necessidade de testes automatizados e integração com Syncthing	Ausência de suíte de testes; dificuldade de sincronizar <code>`~/.emacs.d`</code> entre desktop Android e proot-distro.

✓ Requisitos (Funcionais + Não Funcionais) Extraídos do PDF

🖥️ UI/UX & Desktop

- Toolbar com ícones **grandes e estéticos**, ativável/desativável via macro.
- Página inicial personalizada (``dashboard``) com:
 - Múltiplos desktops/contextos/projetos (ex: ``perspective-el``).
 - Informações atualizadas (clima, tarefas, commits, cotações).
 - Notificações integradas.
- Suporte a ícones em:
 - ``dired`` (local e **via SSH**)
 - ``treemacs`` (ícones coloridos)
 - ``all-the-icons`` + ``nerd-icons`` + ``emojify`` (globais)
- **Doom modeline** ativo e estilizado.
- Tema consistente (ex: ``modus-operandi``, ``doom-one``).

🧩 Integrações e Funcionalidades

- ``vterm`` funcional (shell dentro do Emacs).
- Clipboard integrado com Termux (``termux-clipboard-set/get``).
- ``pdf-tools`` compilado com suporte a imagens e anotações.
- ``bookmark+`` instalado sem erros.
- **Macro ``C-c C-p p``** (Markdown → PDF) funcionando (via ``pandoc`` ou ``org-mode``).
- Todas as outras macros mencionadas (ainda não testadas) devem ser validadas.
- Caminho de autosaves corrigido (remoção do traço ``-``).
- Gerenciamento correto de janelas: alternância entre ``neotree``/``treemacs``, fechamento de painéis, ``C-x 1`` maximizando.

↺ Sincronização e Estrutura

- **Syncthing** configurado para sincronizar `~/emacs.d/` entre:
 - Desktop Ubuntu (via `initAntiGrav.el`)
 - Termux (proot-distro)
 - *Solução*: criar symlink da pasta proot para `/sdcard/emacs-config` e apontar o app Android para lá.
- Módulos `.el` separados em `~/elisp/`, importados pelo `init.el` com `(load-file "~/elisp/...")`.
- Integração seletiva de `~/initAntiGrav.el` (evitar duplicação de funções).

Testabilidade e Documentação

- Testes unitários (ERT), integração, sistema, regressão e **end-to-end** com fixtures e test doubles.
- Gap analysis dos arquivos na pasta `/home/sukata/emacs` (listados no PDF) – extrair requisitos não implementados.
- Conversão de chatlogs (`session-ses\_2716.md`, `session-ses\_295b.md`, `emacsproj.md`) para `.org` com code highlighting e indentação JSON.
- Atualização do `emacs-manual.md` com:
 - Instalação de dependências no Termux (incluindo `sudo` em proot).
 - Configuração de fontes (Nerd Fonts, emoji fallback).
 - Troubleshooting dos erros observados.
 - Lista de macros funcionais e atalhos.

Plano de Correção (do mais simples ao mais complexo)

Fase 1 – Estabilização do Ambiente Básico

1. Permissões e dependências do sistema (proot-distro)

bash



```
apt update && apt install sudo
sudo apt install build-essential cmake pkg-config libpng-dev libpoppler-dev
```



2. Corrigir `init.el`

- Verificar fechamento de parênteses (usar `M-x check-parens`).
- Corrigir variável de autosaves:

elisp

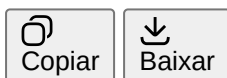


```
(setq auto-save-list-file-prefix (expand-file-name "autosaves/" user-em
```



3. Instalar e compilar `vterm` corretamente

elisp



```
(use-package vterm :ensure t :config (setq vterm-module-cmake-args "-DUSE_SY
```



4. Instalar `bookmark+` via MELPA (ou usar `bookmark.el` nativo temporariamente).

Fase 2 – Ícones, Fontes e Modeline

1. Instalar Nerd Fonts no Termux (ex: `Hack Nerd Font`) e configurar no terminal.

2. Configurar fallback para emojis

elisp



```
(set-fontset-font t 'symbol (font-spec :family "Noto Color Emoji"))
```



3. Instalar e ativar

- ``all-the-icons`, `nerd-icons`, `emojify``
- ``doom-modeline``

4. Forçar ícones via SSH

- Verificar se o terminal remoto suporta Nerd Fonts (ex: usar ``MobaXterm`, `Kitty``).
- No Emacs remoto, definir ``(set-fontset-font t 'symbol (font-spec :family "Hack Nerd Font"))``.

Fase 3 – Toolbar, Dashboard e Desktops

1. Toolbar customizada (macro para toggle):

elisp



```
(defmacro toggle-toolbar () '(tool-bar-mode (if tool-bar-mode -1 1)))
(global-set-key (kbd "C-c t") 'toggle-toolbar)
```



2. Dashboard (``emacs-dashboard``) com widgets personalizados.

3. Múltiplos desktops (``perspective-el`` ou ``workgroups2``).

Fase 4 – Correção de Macros e PDF Export

1. Testar macro ``C-c C-p p``

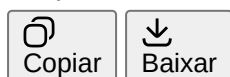
- Instalar ``pandoc`` e ``texlive`` (no proot: ``sudo apt install pandoc texlive``).
- Verificar configuração do ``org-mode`` para exportação PDF via LaTeX.

2. Criar suíte de testes ERT para todas as macros listadas.

Fase 5 – Gerenciamento de Janelas (Neotree vs Treemacs)

- Escolher **apenas um** (recomendo ``treemacs``).
- Desabilitar ``neotree`` e mapear teclas para fechar painel:

elisp




```
(global-set-key (kbd "C-x t") 'treemacs)
(global-set-key (kbd "C-x 1") 'delete-other-windows)
```

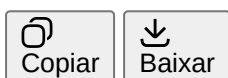


3. Garantir que `C-x 1` maximize a janela atual corretamente.

Fase 6 – Integração e Sincronização

1. **Syncthing** no Android: criar pasta sincronizada em `/sdcard/emacs-sync`.
2. No proot-distro, criar symlink:

bash



```
ln -s /sdcard/emacs-sync ~/.emacs.d
```



3. Integrar `initAntiGrav.el` (copiar funções não conflitantes para `~/elisp/`).

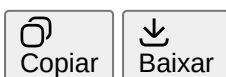
Fase 7 – Testes e Documentação

1. **Testes ERT** para:
 - Clipboard
 - Macros (MD → PDF, toggle toolbar, etc.)
 - Abertura de arquivos no diretd/treemacs
2. **Gap analysis** dos arquivos PDF/MD da pasta `/home/sukata/emacs` – mapear requisitos teóricos vs. implementados.
3. **Converter chatlogs** para `.org` com syntax highlighting (ex: `pandoc chatlog.md -o chatlog.org`).
4. **Atualizar `emacs-manual.md`** com todas as correções e instruções.



Pacotes Essenciais (Termux – proot-distro Ubuntu)

bash



```

sudo apt update
sudo apt install -y emacs build-essential cmake pkg-config \
  libpng-dev libpoppler-dev libvterm-dev pandoc texlive \
  git curl unzip
# Nerd Fonts
curl -L -o ~/.local/share/fonts/Hack.zip https://github.com/ryanoasis/nerd-fonts/
unzip ~/.local/share/fonts/Hack.zip -d ~/.local/share/fonts/
fc-cache -fv

```



Exemplo de Teste ERT para Validação da Macro

MD → PDF

elisp



Copiar



Baixar

```

(ert-deftest test-md-to-pdf ()
  (let ((temp-md (make-temp-file "test" nil ".md" "# Heading\nContent"))))
    (with-current-buffer (find-file-noselect temp-md)
      (markdown-mode)
      (call-interactively 'my-md-to-pdf) ; ou a macro real
      (should (file-exists-p (concat (file-name-sans-extension temp-md) ".pdf")))
      (delete-file (concat (file-name-sans-extension temp-md) ".pdf"))
      (kill-buffer))))

```



Organização de Chatlogs no OpenCode – Sugestão

Para evitar confusão em conversas com múltiplas tarefas:

1. **Use tópicos com `###` e emojis para separar contextos.**
2. **Ao iniciar uma nova seção**, referencie o chatlog anterior com `@session-ses\_2716.md` e use `/resume`.
3. **Converta snippets JSON para blocos indentados** com ````json` no Markdown.
4. **Exporte o histórico final para `.org`** para melhor navegação e code highlighting.

Exemplo de estrutura:

org



- * 📌 Correção de vterm
 - Problema: compilação falha
 - Solução: instalar libvterm-dev

- * 🎨 Ícones via SSH
 - Diagnóstico: fonte Nerd Fonts não carregada
 - Correção: set-fontset-font



Rationale Final

Este prompt foi construído para **resolver todas as falhas documentadas no PDF** de forma incremental, garantindo que cada correção seja testável e documentada. A prioridade é:

- 1 Estabilidade (permissões, init.el, autosaves)
- 2 Funcionalidades visuais (ícones, toolbars)
- 3 Macros e exportação (MD → PDF)
- 4 Sincronização e testes

Com isso, o OpenCode pode executar as tarefas de maneira **coesa, concisa, conectada e coerente**, gerando um ambiente Emacs plenamente funcional no Termux/proot-distro.